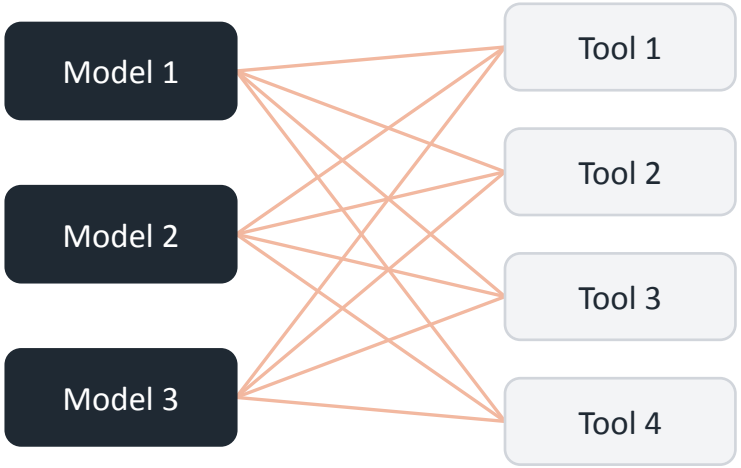


THE UNIVERSITY OF CHICAGO
DATA SCIENCE INSTITUTE

How Agents Act: MCP and Globus Compute

The problem with tool calling

- ✗ N models × M tools = N×M integrations
- ✗ Every team writes their own glue code
- ✗ No standard auth, schema, or transport
- ✗ Tool changes break every integration



3 models × 4 tools = 12 custom integrations

What is MCP?

- Open standard for connecting AI apps to external tools and data
- Solves the N×M problem: one server per integration that works with any MCP-compatible app
- Client-server architecture: harness (app) hosts a client, and tools are exposed by servers
- Transport-agnostic (e.g., local and remote)
- Tools self-describe with a name, description, and JSON schema

Broad adoption: Anthropic, OpenAI, Google, Microsoft, and major agent frameworks, now governed by the Linux Foundation

https://modelcontextprotocol.io

Model (Context Protocol) - Intelligence layer

- The model reads tool schemas and decides when and how to call them
- It emits `tool_use/tool_call` blocks instead of text when it wants to act
- It resumes generation once results return to the context

→ Good schema design is about communicating intent to the model (e.g., think about documentation for a colleague with no other context)

(Model) **Context** (Protocol) - Memory layer

- The context window is everything the model knows for that session:
  - System prompt, conversation history, tool calls, results, …
- Append-only and finite
- Tool results can be large

(Model Context) **Protocol** - Communication Layer

- Standard JSON-RPC over stdio (local) or HTTP (remote)
- Three primitives:
  - Tools (model-driven): capabilities AI model decides to call
  - Resources (app-driven): read-only data the app decides to put in context
  - Prompts (user-driven): reusable templates selected by users
- Auth: env vars for local access, OAuth for SaaS, API keys for service-to-service

MCP primitives

Tools

`@mcp.tool`

- Functions the model chooses to call
- Can take actions and have side effects
- Supported by every MCP client

Resources

`@mcp.resource`

- Read-only data: files, records, configs
- The app or user decides what goes into context
- Addressed by URI, e.g., data://constants/{name}

Prompts

`@mcp.prompt`

- Reusable prompt templates
- The user explicitly picks them, often as slash commands
- Take arguments to fill in the template

Tool call lifecycle

- App obtains the tool list and schemas at the start of the session
- When the model decides to act, it emits a **`tool_use`** block instead of text
- Agent detects **`tool_use`** block, extracts tool name and arguments
- Agent calls the MCP server with those arguments
- Server executes and returns a result
- Result is appended to the conversation as a **`tool_result`** message
- Model resumes generation with the result now in context
- Loop continues until the model emits plain text with no tool call

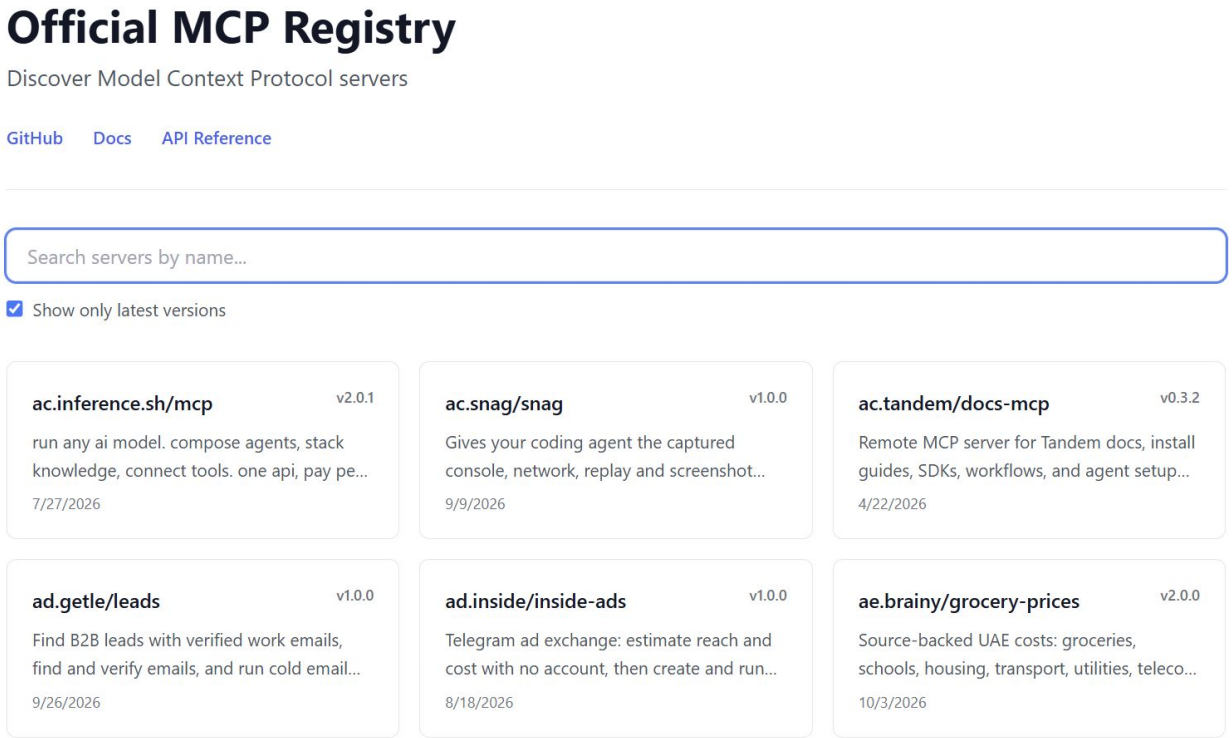
A word about context

- Every tool call and every result gets appended to the context window for that session
- Tool results can be large (e.g., reading a large codebase)
- Strategies:
  - Summarize results before returning them, use resources for large reads so the app controls what is loaded
  - Design tools to return answers (not raw data)
  - Set token budget and limit the number of tool calls per session
  - Watch for runaway loops

Tool schema design

- The model reads schema to decide whether, when, and how to call a tool
  - Schemas are very important → bad schemas will lead to broken calls
- The description should say when to use it and when not to
  - Vague descriptions → model calling at the wrong time with wrong args
- Give parameters a description (hard to tell from a name)
- Use *enum* for constrained choices
- State side effects explicitly: "this will send an email", "this deletes permanently"
- Think of the schema as a contract between you and the model

Finding an MCP server



https://registry.modelcontextprotocol.io/

Activity 5: Building an MCP Server

- Goal: build an MCP server, expose 2-3 tools, connect your agent to it



https://gofastmcp.com/

1. Setup

- Install: `pip install fastmcp`
- Complete `04_server.py`
- Start it: `python 04_server.py (or fastmcp run server.py --transport http --port 8765)`
- Verify: `list` and confirm your server responds
  - `fastmcp list http://localhost:8765/mcp`

THE UNIVERSITY OF CHICAGO
DATA SCIENCE INSTITUTE

Basic FastMCP Server

```python
from fastmcp import FastMCP
mcp = FastMCP("Demo 🚀")

@mcp.tool
def add(a: int, b: int) -> int:
    """Add two numbers."""
    return a + b

if __name__ == "__main__":
    mcp.run(transport="http", port=8765)
```

2. Expose your Tools

- Add 2-3 tools using the `@mcp.tool` decorator
  a. For example: simulation, calculation, data access
- Use type hints and docstrings (to generate the schema automatically)
- Run `list` again
- Checkpoint: does the schema description match what you'd want the model to read?

3. Connect the agent

- Point your client (`04_client.py`) at the local server URL
- Ask it something that requires using a tool:
  - "*estimate pi with different numbers of samples*"
- Confirm the model chose the right tool with the right arguments

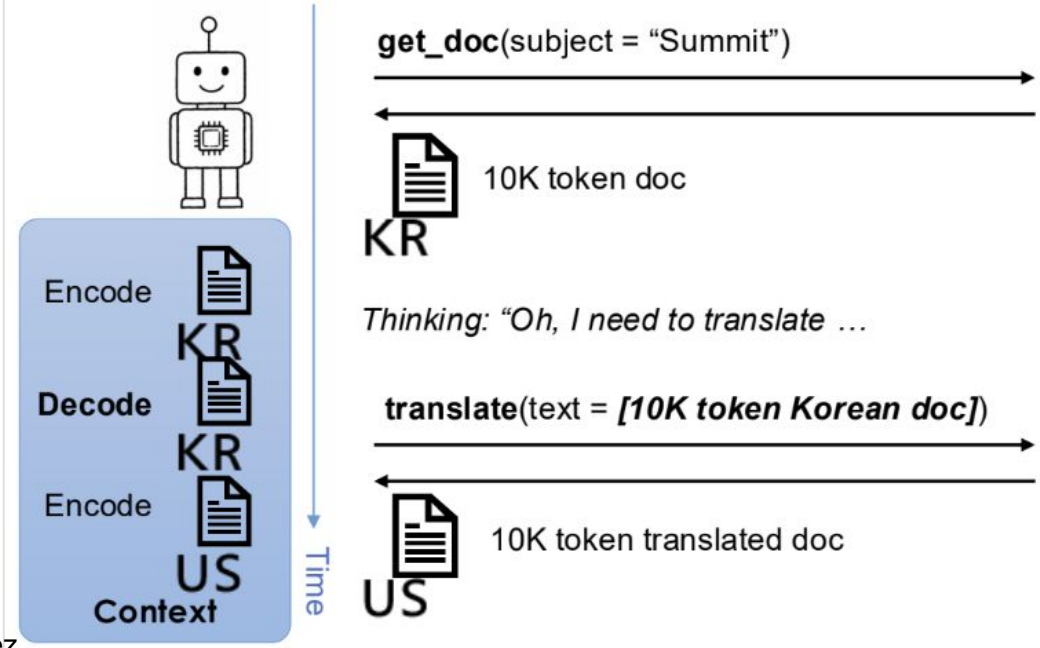
- Checkpoint: did the model call the tool you expected, or did it guess wrong? If wrong — fix the schema
- Advanced: add a resource and a prompt that can improve agent performance

Limitations of MCP

- Context pressure: large tool results fill the context window and degrade performance
- Stateless: servers forget everything between sessions
- Latency: every sequential tool call is a round trip
- The model can be wrong: wrong tool, bad arguments, or unnecessary loops
- Descriptions: vague tool descriptions cause unpredictable calls
- No transactions: a failed step leaves partial side effects with no rollback
- Hard to debug: tracing why the model made a call needs extra tooling
- Still early: the spec is evolving and server quality varies

Scalable tool calling: MCP to code execution

Today: Standard Tool Calling



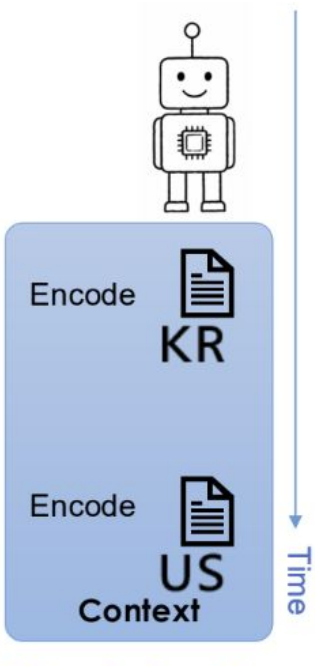


Large amounts of **data pass through model context** between tool calls.

Slide: Joseph Gonzalez, LBNL Agents Workshop

Scalable tool calling: MCP to code execution

Programmatic Tool Calling (**PTC**)*




```
import tools
doc = tools.get_doc(subject = "Summit")
print(doc)
```

10K token doc

KR

Thinking: "Oh, I need to translate …

```
import tools
doc = tools.get_doc(subject = "Summit")
doc = tools.translate(text = doc)
print(doc)
```

US

Use code to **call** and ***compose*** **tools**

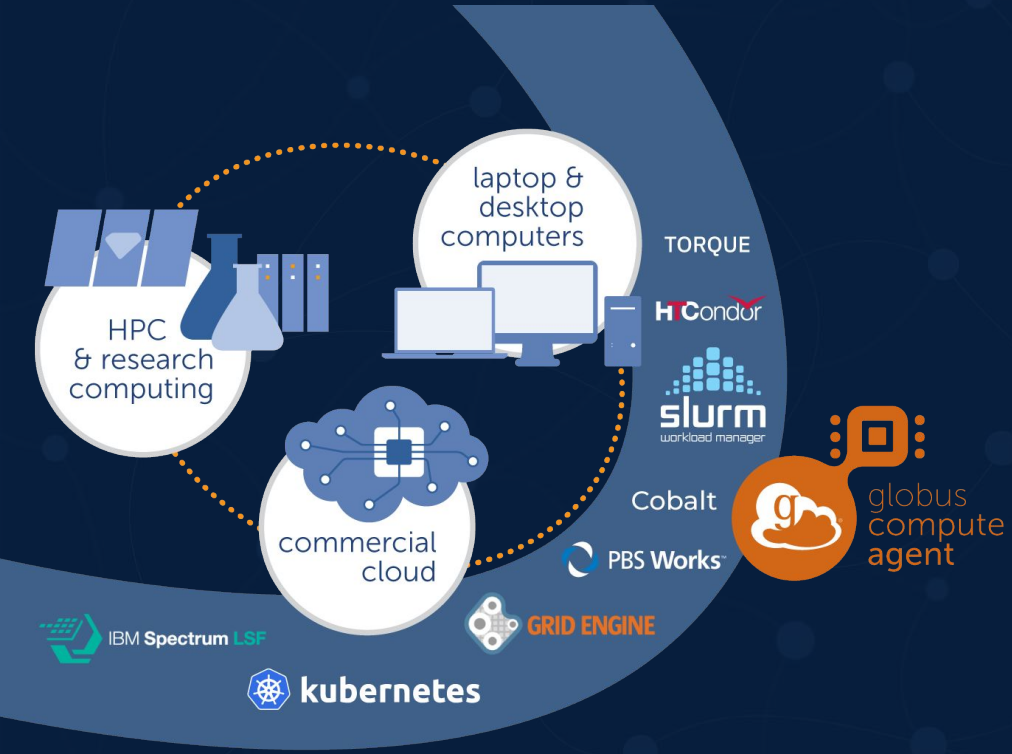


Fails to **reuse accumulated state** across program calls
→ **Repeated Work**

*Introducing advanced tool use on the Claude Developer Platform

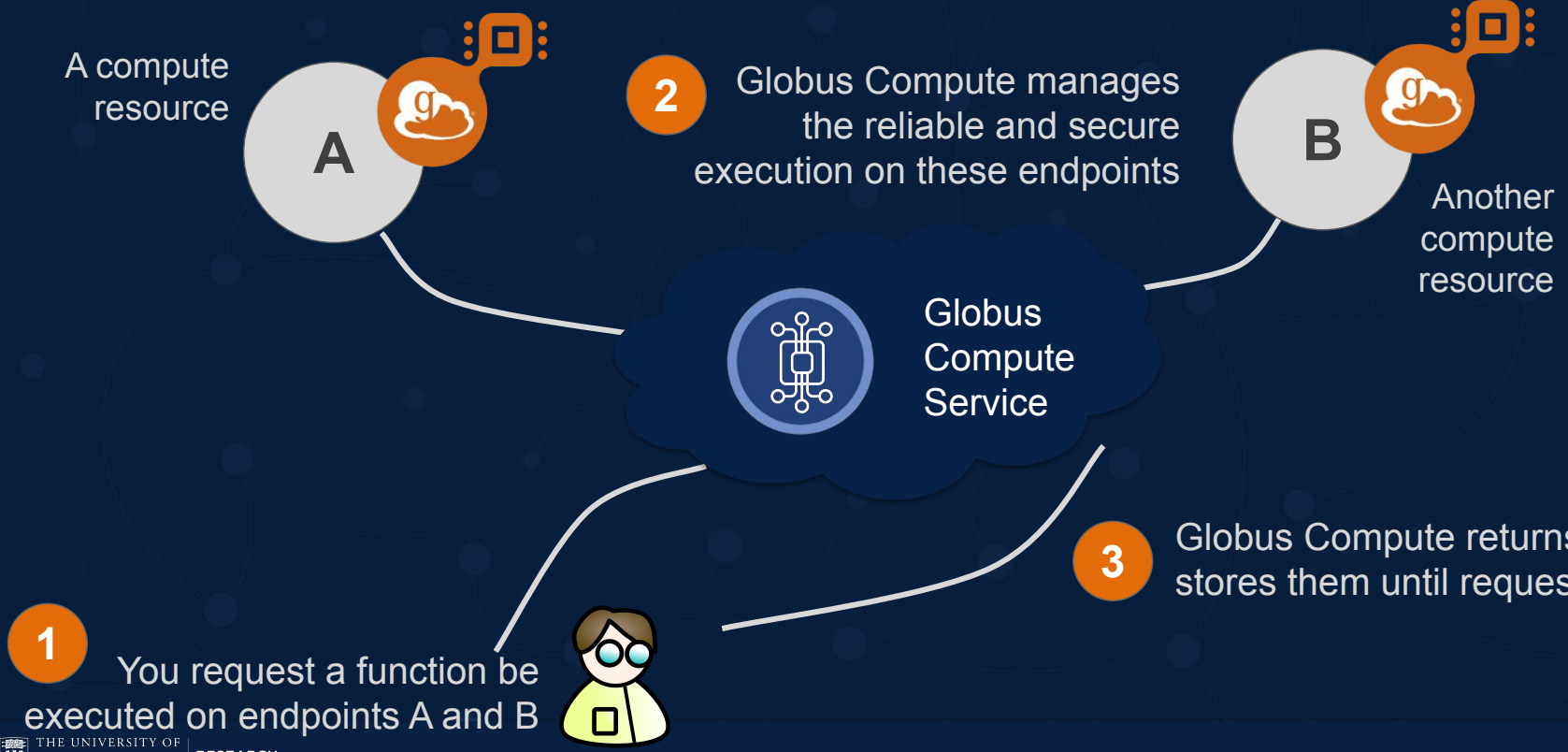
Slide: Joseph Gonzalez, LBNL Agents Workshop

Globus Compute - Managed Federated FaaS



- FaaS for any compute resource
- Programmatic access to compute resources
- Consistent user interface across diverse execution systems
- “Fire and forget” reliable execution
- Rich multi-level policy specification & enforcement

What does it look like from the researcher's PoV?



Option 1: Create our own endpoint

```
$ uv tool install globus-compute-endpoint
$ globus-compute-endpoint configure <ENDPOINT_NAME>
$ globus-compute-endpoint start <ENDPOINT_NAME>
```

```
$ export GLOBUS_COMPUTE_ENDPOINT=<ENDPOINT_ID>
```

Option 2: Use the Polaris endpoint

```
$ export GLOBUS_COMPUTE_ENDPOINT=<ENDPOINT_ID>
```

```
POLARIS_CONFIG = {
    'account': 'ANL-FermiAgenticTraining',
    'queue': 'debug',
}

gce = Executor(endpoint_id=compute_endpoint,
               user_endpoint_config=POLARIS_CONFIG)
```

References

Model Context Protocol: https://modelcontextprotocol.io

FastMCP: https://gofastmcp.com

Globus Compute: https://globus-compute.readthedocs.io

ALCF guide to Globus Compute on Polaris:
https://docs.alcf.anl.gov/services/globus-compute/